GranuWarm: Step-by-Step Learning & Implementation Roadmap

A Phase-by-Phase Mastery Guide for CSE Research Students

Welcome to the official step-by-step learning and implementation roadmap for **GranuWarm**! This guide breaks down the full system into 15 manageable, self-contained phases (Phase 0 through Phase 14).

> **Rule of Engagement:** Complete and verify **one phase at a time**. Do not attempt to write code for future phases until the current phase's verification criteria are fully met.

PHASE 0 — Prerequisites

1. What You Need to Learn

- Linux Command Line Interface (Bash/Zsh navigation, process management, environment variables).
- Python 3.11 core fundamentals (`asyncio`, type hinting, package management via `pip`, virtual environments via `venv`).
- Version control with Git (`git init`, `git commit`, `.gitignore` management).
- Basic computer systems concepts (CPU, RAM, process threads, HTTP/REST basics).

2. Why You Need It

GranuWarm is an asynchronous Python application that controls containerized Linux workloads. A solid foundation in Linux, Python 3.11, and Git prevents environment issues later.

3. Exact Practical Task

1. Set up an Ubuntu 22.04 LTS development environment (native, dual-boot, or VM with at least 8 CPU cores and 16 GB RAM).
2. Install base system utilities:

```
sudo apt update && sudo apt install -y curl git htop jq build-essential
python3-pip python3-venv
```

3. Initialize the GranuWarm Git repository and create a clean Python 3.11 virtual environment:

```
mkdir granuwarm && cd granuwarm
git init
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

4. Expected Output

A clean Linux shell with an active `(venv)` prompt and verified dependencies.

5. How to Verify It Works

Run the following verification commands:

```
python3 --version   # Must output >= 3.11
git --version       # Must output git version
jq --version        # Must output jq version
```

6. Common Errors

- **Python version < 3.10:** Ensure Ubuntu package repository references Python 3.11+.
- `permission denied` **during package installation:** Do not run `pip install` with `sudo` while inside a virtual environment.

7. Relevant GitHub Repositories

- `vinta/awesome-python` — Python environment & library guide.
- `jlevy/the-art-of-command-line` — Command line fundamentals.

8. When to Move to the Next Phase

Move to Phase 1 once your Linux virtual environment is active, verified, and clean.

PHASE 1 — Docker

1. What You Need to Learn

- Containerization principles and OS process isolation (Namespaces and cgroups).
- `Dockerfile` syntax (`FROM`, `WORKDIR`, `COPY`, `RUN`, `CMD`).
- Docker CLI commands (`docker build`, `run`, `ps`, `logs`, `stop`, `exec`).
- Multi-layer image architecture (OverlayFS) and cache optimization.

2. Why You Need It

GranuWarm's benchmark functions ($F_1 \dots F_6$) must run inside standardized, reproducible container images across Python 3.11 and Node.js 20 runtimes.

3. Exact Practical Task

1. Install Docker Engine v26+ on Ubuntu:

```
sudo apt install -y docker.io
sudo usermod -aG docker $USER
newgrp docker
```

2. Create `docker/base_python/Dockerfile` and `docker/base_node/Dockerfile`.
3. Create minimal function code for `F1` (Node.js Hello) and `F3` (Python Image Processing).
4. Build local Docker images:

```
docker build -t granuwarm/f1_hello:v1 functions/f1_hello/
docker build -t granuwarm/f3_image_resize:v1 functions/f3_image_resize/
```

4. Expected Output

Two container images (`granuwarm/f1_hello:v1` and `granuwarm/f3_image_resize:v1`) built and listed in local Docker image storage.

5. How to Verify It Works

Run a container locally and test its HTTP endpoint:

```
docker run -d --name test_f1 -p 8081:8080 granuwarm/f1_hello:v1
curl http://localhost:8081
docker stop test_f1 && docker rm test_f1
```

Expected response: `{"message": "Hello from F1!"}`.

6. Common Errors

- **`permission denied while trying to connect to the Docker daemon socket`:** Run `sudo usermod -aG docker $USER` and log out/in.
- **Large image size (> 1 GB):** Ensure base images use slim variants (e.g., `python:3.11-slim`, `node:20-slim`) and clean build caches (`pip --no-cache-dir`).

7. Relevant GitHub Repositories

- `docker/awesome-docker` — Curated Docker resources.
- `wsargent/docker-cheat-sheet` — Docker CLI guide.

8. When to Move to the Next Phase

Move to Phase 2 once you can build Docker images and curl their HTTP endpoints locally.

PHASE 2 — Kubernetes (K8s)

1. What You Need to Learn

- Core Kubernetes abstractions: Nodes, Pods, Deployments, Services, Namespaces, ConfigMaps.
- `kubectl` CLI commands (`get`, `describe`, `apply`, `logs`, `delete`).
- Structure and syntax of declarative Kubernetes YAML manifests.

2. Why You Need It

Kubernetes is the standard orchestrator for production serverless infrastructure. Knative and GranuWarm operate directly inside a Kubernetes cluster.

3. Exact Practical Task

1. Install `kubectl` CLI utility on your host machine:

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

2. Study basic K8s YAML manifest structures for Deployments and Services.
3. Understand how Kubernetes uses Label Selectors to route traffic from a Service to underlying Pods.

4. Expected Output

`kubectl` CLI tool installed and verified on host. Conceptual understanding of K8s Pod and Service networking.

5. How to Verify It Works

Run `kubectl version --client` to confirm client binary installation.

6. Common Errors

- **Confusing Containers with Pods:** A Pod is a wrapper that holds one or more containers sharing network IP and volumes.
- **YAML Syntax Errors:** Indentation must use spaces, never tabs.

7. Relevant GitHub Repositories

- `kubernetes/kubernetes` — Official K8s repository.
- `ramitsurana/awesome-kubernetes` — Kubernetes learning guides.

8. When to Move to the Next Phase

Move to Phase 3 once `kubectl` is installed and you can explain the structural difference between a K8s Deployment and a K8s Service.

PHASE 3 — KinD (Kubernetes-in-Docker)

1. What You Need to Learn

- How KinD provisions local multi-node Kubernetes clusters inside Docker containers.
- Mapping ingress ports (80/443) from host machine to KinD node containers.
- Loading local Docker images directly into KinD nodes without pushing to external registries (`kind load docker-image`).

2. Why You Need It

KinD allows running a local Kubernetes cluster on a single developer host with zero cloud costs and 100% test environment reproducibility.

3. Exact Practical Task

1. Install KinD v0.22+:

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.22.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

2. Create `configs/cluster_config.yaml` specifying node configuration and port mappings.
3. Launch cluster: `kind create cluster --config configs/cluster_config.yaml --name granuwarm-local`.
4. Load local Docker images into KinD:

```
kind load docker-image granuwarm/f1_hello:v1 --name granuwarm-local
```

4. Expected Output

A running KinD Kubernetes cluster named `granuwarm-local` accessible via `kubectl`.

5. How to Verify It Works

```
kubectl cluster-info --context kind-granuwarm-local
kubectl get nodes
```

Expected output: Node `granuwarm-local-control-plane` status `Ready`.

6. Common Errors

- **Docker service not running:** Ensure Docker daemon is active (`sudo systemctl status docker`).
- **`ImagePullBackOff` in cluster:** KinD nodes cannot see host Docker images unless you explicitly execute `kind load docker-image <image_name> --name granuwarm-local`.

7. Relevant GitHub Repositories

- `kubernetes-sigs/kind` — Official KinD repository.

8. When to Move to the Next Phase

Move to Phase 4 when `kubectl get nodes` returns a `Ready` control plane node inside KinD.

PHASE 4 — Knative

1. What You Need to Learn

- Knative Serving core architecture (Knative Service CRDs, Revisions, Routes).
- Knative Pod Autoscaler (KPA) and scale-to-zero mechanism.
- Knative Kourier Ingress routing and domain configuration (`sslip.io`).
- Controlling minimum scale using the `autoscaling.knative.dev/min-scale` annotation.

2. Why You Need It

Knative supplies the serverless Function-as-a-Service runtime environment that GranuWarm optimizes.

3. Exact Practical Task

1. Write setup script `scripts/setup_env.sh` to install Knative Serving v1.13+ and Kourier ingress onto KinD.
2. Configure Knative default domain to `127.0.0.1.sslip.io`.
3. Create Knative Service YAML manifests (`service.yaml`) for functions $F_1 \dots F_6$.
4. Deploy $F_1 \dots F_6$ to Knative: `kubectl apply -f functions/`.

4. Expected Output

6 serverless functions deployed to Knative, automatically scaling to 0 when idle and scaling up upon incoming HTTP requests.

5. How to Verify It Works

```
kubectl get ksvc
curl -H "Host: f1-hello.default.127.0.0.1.sslip.io" http://localhost:80
```

Verify that the initial call incurs a ~1s cold-start delay, subsequent calls return instantly (~5ms), and after 60s of silence, pods scale back to 0 (`kubectl get pods`).

6. Common Errors

- **Kourier Ingress Pending IP:** Ensure host port 80 is mapped correctly in `configs/cluster_config.yaml`.
- **DNS lookup failures:** Use `127.0.0.1.sslip.io` magic DNS domain to avoid custom `/etc/hosts` editing.

7. Relevant GitHub Repositories

- `knative/serving` — Official Knative Serving repository.
- `knative/docs` — Knative documentation & code samples.

8. When to Move to the Next Phase

Move to Phase 5 once all 6 benchmark functions deploy to Knative, handle HTTP requests, and auto-scale to zero.

PHASE 5 — Prometheus

1. What You Need to Learn

- Time-series metric collection principles (Pull model, scrape intervals).
- Deploying applications using Helm chart package manager.
- PromQL syntax (`sum`, `rate`, `increase`, `offset`) for computing rates and derivatives over sliding time windows.

2. Why You Need It

Prometheus is GranuWarm's operational sensor network, extracting live measurements (R_i, T_i, C_i, M) from the cluster.

3. Exact Practical Task

1. Install Helm v3 on your host:

   ```
   curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
   ```

2. Install Prometheus into `monitoring` namespace using Helm via `scripts/setup_env.sh`.
3. Implement `controller/monitor/prometheus_client.py` using Python `requests` or `prometheus-api-client` to execute PromQL queries.

4. Expected Output

Prometheus running on cluster, scraping Knative operational metrics every 2s, and a Python client returning structured signal dictionaries.

5. How to Verify It Works

Run a standalone test script invoking `prometheus_client.py`:

```
# Test Prometheus Client Extraction
from controller.monitor.prometheus_client import PrometheusMonitor
pm = PrometheusMonitor(prometheus_url="http://localhost:9090")
metrics = pm.get_function_metrics("f1-hello")
print(metrics) # Expected: {'R_i': 0.0, 'T_i': 0.0, 'C_i': 0.0, 'M': 0.12}
```

6. Common Errors

- **Empty PromQL query response:** Ensure target Knative metric labels match exact query filter tags (`service_name="f1-hello"`).
- **Scrape interval latency:** Scrape interval must be set to 2s in Prometheus ConfigMap to support 10s fast-loop reactive control.

7. Relevant GitHub Repositories

- `prometheus/prometheus` — Official Prometheus repository.
- `prometheus-community/helm-charts` — Helm charts for Prometheus.

8. When to Move to the Next Phase

Move to Phase 6 when your Python client successfully queries live PromQL metric streams from Prometheus.

PHASE 6 — Workload Generator

1. What You Need to Learn

- Asynchronous HTTP programming in Python using `asyncio` and `httpx`.
- Mathematical workload rate modeling (Poisson arrivals, linear growth, burst spikes, diurnal sine waves).
- Command-line argument parsing with Python `argparse`.

2. Why You Need It

To evaluate GranuWarm's adaptive prewarming performance under reproducible traffic regimes ($W_1 \dots W_5$).

3. Exact Practical Task

1. Implement `workload/profiles.py` defining rate functions $\lambda_i(t)$ for profiles $W_1 \dots W_5$.
2. Implement `workload/generator.py` using `asyncio` + `httpx.AsyncClient` to fire non-blocking HTTP requests at Knative endpoints.
3. Support configurable random seeds (`--seed`) for reproducibility.

4. Expected Output

A CLI workload generator able to stream HTTP traffic targeting Knative functions matching selected profiles ($W_1 \dots W_5$).

5. How to Verify It Works

Run generator for 60 seconds on profile W_3 (Bursty):

```
python workload/generator.py --profile W3 --duration 60 --seed 42
```

Check Prometheus web UI (`http://localhost:9090`) to observe request rate spikes on graph dashboard.

6. Common Errors

- **Client-side bottlenecking:** Using synchronous `requests` instead of asynchronous `httpx` limits maximum generated throughput.
- **Socket exhaustion:** Re-use a single `httpx.AsyncClient` session instance instead of creating new client objects per request.

7. Relevant GitHub Repositories

- `encode/httpx` — Asynchronous HTTP client for Python.
- `locustio/locust` — Python load testing framework.

8. When to Move to the Next Phase

Move to Phase 7 when the workload generator reproduces all 5 workload shapes ($W_1 \dots W_5$) visible in Prometheus graphs.

PHASE 7 — Simulator

1. What You Need to Learn

- Discrete-event simulation concepts in pure Python.
- Synthetic execution modeling (warm start vs cold start delays, memory footprint tracking).
- Object-oriented state representation for FaaS container lifecycles.

2. Why You Need It

The simulator allows instant offline debugging and tuning of GranuWarm logic in seconds without waiting for 10-minute live Kubernetes cluster runs.

3. Exact Practical Task

1. Implement `simulator/env.py` simulating container state, queue delay, warm hits, cold start penalties, and memory usage.
2. Implement `simulator/run_sim.py` to execute synthetic workload traces through candidate control algorithms.

4. Expected Output

A standalone Python simulation script executing 1000 seconds of workload simulation in under 3 seconds real time, outputting summary latency and memory metrics.

5. How to Verify It Works

Run simulation script and inspect generated execution logs:

```
python simulator/run_sim.py --workload W3 --policy fixed
```

Verify output CSV log contains columns `timestamp`, `function_id`, `latency`, `is_cold_start`, and `idle_memory`.

6. Common Errors

- **Time step drift:** Ensure simulated clock advances deterministically in fixed discrete intervals Δt.
- **Inconsistent warm/cold latency values:** Align synthetic function execution delays with benchmark measurements ($\tau_i^{warm}, \tau_i^{cold}$).

7. Relevant GitHub Repositories

- `simpy/simpy` — Discrete-event simulation framework for Python.

8. When to Move to the Next Phase

Move to Phase 8 when your Python simulator reliably executes offline workloads and outputs clean performance metrics.

PHASE 8 — Fuzzy Controller

1. What You Need to Learn

- Mamdani Fuzzy Inference System (FIS) theory (Fuzzification, Rule evaluation, Aggregation, Defuzzification).
- Fuzzy membership function modeling (Triangular $trimf$, Trapezoidal $trapmf$).
- Centroid defuzzification math.
- Using `scikit-fuzzy` or custom NumPy matrix operations.

2. Why You Need It

The Fuzzy Controller converts noisy workload signals (R_i, T_i, C_i, M) into robust target capacity recommendations (P_i^{fuzzy}).

3. Exact Practical Task

1. Create `configs/fuzzy_rules.json` containing the 27 Mamdani rule definitions.
2. Implement `controller/fuzzy/fuzzy_engine.py` evaluating normalized inputs against membership curves.
3. Return scalar continuous warm capacity recommendations $P_i \in [0, 1]$.

4. Expected Output

A modular Python FIS engine mapping input metric quadruples $[R_i, T_i, C_i, M]$ to output P_i^{fuzzy}.

5. How to Verify It Works

Write and run unit tests (`tests/test_fuzzy.py`):

```
# Verify Surging Traffic triggers High Capacity recommendation
def test_fuzzy_surging():
    engine = FuzzyEngine()
    # Input: [Intensity=0.3, Trend=+0.5 (Surging), ColdStart=0.2, Memory=0.5]
```

```
    p_target = engine.evaluate(R=0.3, T=0.5, C=0.2, M=0.5)
    assert p_target >= 0.70  # Target capacity must be >= 70%
```

6. Common Errors

- **Divide-by-zero during defuzzification:** Occurs if input combination fires zero rules. Ensure fuzzy set memberships overlap continuously over universe domain $[0, 1]$.
- **Inverted triangular parameters:** Triangular function parameters must satisfy $a \leq b \leq c$.

7. Relevant GitHub Repositories

- `scikit-fuzzy/scikit-fuzzy` — Fuzzy logic toolkit for SciPy.

8. When to Move to the Next Phase

Move to Phase 9 when unit tests confirm all 27 fuzzy rules execute correctly and return bounded outputs $P_i \in [0, 1]$.

PHASE 9 — Granularity Tree

1. What You Need to Learn

- Tree / Directed Acyclic Graph (DAG) data structures in Python.
- Software dependency decomposition (Base OS, Runtime, Shared Libraries, Application Code).
- Max-projection aggregation algorithms ($P_k^{tree} = \max_{f_i \in \mathcal{F}(v_k)} P_i$).

2. Why You Need It

The Dependency Granularity Tree translates function-level capacity recommendations into shared component prewarming targets, optimizing shared memory footprint.

3. Exact Practical Task

1. Create `configs/dependencies.json` encoding the 6-function dependency hierarchy ($v_0 \dots v_{11}$).
2. Implement `controller/granularity/tree.py` to parse JSON tree structure and perform parent-child path traversals and max-projections.

4. Expected Output

A Python class that ingests function recommendations $[P_1 \dots P_6]$ and produces node target allocation vector P^{tree}.

5. How to Verify It Works

Run unit test (`tests/test_tree.py`):

```
def test_tree_projection():
    tree = GranularityTree("configs/dependencies.json")
    # Inputs: F1 requires 3 instances, F2 requires 1 instance
```

```
    node_targets = tree.project_capacities({"f1": 3, "f2": 1})
    # Node v1 (Shared Node.js Runtime) must inherit max(3, 1) = 3
    assert node_targets["v1"] == 3
```

6. Common Errors

- **Circular references in JSON:** Ensure tree node links are strictly directed and acyclic.
- **Missing leaf mapping:** Verify every benchmark function $F_1 \dots F_6$ maps to a valid leaf node in the tree.

7. Relevant GitHub Repositories

- `networkx/networkx` — Network analysis in Python.

8. When to Move to the Next Phase

Move to Phase 10 when tree projection accurately propagates function capacity values across all dependency nodes.

PHASE 10 — Particle Swarm Optimization (PSO)

1. What You Need to Learn

- Nature-inspired Swarm Intelligence principles.
- Velocity and Position vector update dynamics with inertia weight decay (ω).
- Discrete / Integer decision-space mapping (rounding, domain clamping).
- Multi-objective fitness function design $J(x)$ using quadratic memory penalty terms.

2. Why You Need It

Constrained Discrete PSO performs global multi-objective optimization, finding integer allocation vectors x^* that satisfy strict cluster memory limits.

3. Exact Practical Task

1. Implement `controller/pso/pso_optimizer.py` using vectorized NumPy array operations.
2. Implement multi-objective fitness function $J(x) = w_L L + w_C C + w_M M + w_S S + \text{Penalty}(x)$.
3. Enforce search space boundaries and integer clamping.

4. Expected Output

An optimizer module returning globally optimal integer allocation vectors x^* within 20–30 iterations.

5. How to Verify It Works

Test PSO optimization against known test parameters:

```
def test_pso_convergence():
    pso = DiscretePSO(swarm_size=15, max_iter=20, memory_budget=400)
```

```
    best_x, best_cost = pso.optimize(target_tree_vector=P_tree)
    # Assert total memory consumption <= 400 MB budget
    assert calculate_memory(best_x) <= 400
```

6. Common Errors

- **Swarm divergence:** Ensure particle velocities are clamped to $[-v_{max}, +v_{max}]$.
- **Infeasible solution winning:** If winning particle exceeds memory budget, increase penalty coefficient ρ ($\rho \geq 100.0$).

7. Relevant GitHub Repositories

- `pyswarms/pyswarms` — Swarm intelligence library in Python.

8. When to Move to the Next Phase

Move to Phase 11 when PSO consistently converges to valid integer allocations respecting memory limits.

PHASE 11 — Knative Controller Integration

1. What You Need to Learn

- Official Kubernetes Python Client SDK (`kubernetes-client`).
- Asynchronous dual-timescale event loops in Python (`asyncio`).
- Hysteresis deadband stabilization logic to prevent allocation flapping.
- Patching custom resource annotations via Kubernetes API.

2. Why You Need It

Integrates Workload Monitor, FIS Engine, Granularity Tree, and PSO into an autonomous control loop that dynamically manages live Knative services.

3. Exact Practical Task

1. Implement `controller/allocator/knative_scaler.py` using `kubernetes.client.CustomObjectsApi()` to patch `autoscaling.knative.dev/min-scale`.
2. Implement `controller/main.py` managing fast reactive loop (10s) and slow global optimization loop (60s).
3. Apply hysteresis thresholds before triggering Knative scale-down commands.

4. Expected Output

A live control daemon continuously monitoring Prometheus metrics and patching Knative service instance bounds.

5. How to Verify It Works

Launch controller alongside workload generator:

```
python controller/main.py &
python workload/generator.py --profile W3 --duration 120
```

In a second terminal, monitor Knative service scaling live:

```
kubectl get ksvc -w
```

Observe `autoscaling.knative.dev/min-scale` annotations adjusting dynamically in response to traffic spikes.

6. Common Errors

- **RBAC / API permission errors:** Ensure `kubeconfig` context is loaded correctly by Python SDK (`config.load_kube_config()`).
- **Scaling oscillation ("Flapping"):** Ensure scale-down requires dropping below current allocation by more than hysteresis threshold δ_{hyst}.

7. Relevant GitHub Repositories

- `kubernetes-client/python` — Official K8s Python SDK.

8. When to Move to the Next Phase

Move to Phase 12 when the controller runs continuously on live Kubernetes for 30+ minutes without crashes or flapping.

PHASE 12 — Baselines

1. What You Need to Learn

- Modular software design for comparative algorithm evaluation.
- Implementing baseline models: B1 (No Prewarm), B2 (Fixed), B3 (Fuzzy-Only), B4 (PSO-Only), B5 (Fuzzy+PSO No Granularity), B6 (Full GranuWarm).

2. Why You Need It

Comparative baselines are required to scientifically prove GranuWarm's advantages and perform ablation studies (RQ1–RQ3).

3. Exact Practical Task

1. Create modular algorithm wrappers inside `baselines/` directory (`no_prewarm.py`, `fixed_prewarm.py`, `fuzzy_only.py`, `pso_only.py`, `no_granularity.py`).
2. Ensure all baseline modules conform to a unified execution interface (`evaluate_allocation()`).

4. Expected Output

6 executable baseline algorithms ready for automated experimental evaluation.

5. How to Verify It Works

Execute each baseline on simulator for 100 steps and confirm expected behavior:

- B1 keeps `minScale = 0`.
- B2 keeps `minScale = 2`.
- B3 updates allocations using Fuzzy logic without PSO optimization.

6. Common Errors

- **Leaking state across baseline runs:** Reset all controller registers, historical averages, and metrics when switching baselines.

7. Relevant GitHub Repositories

- `vinta/awesome-python` — Modular Python software design patterns.

8. When to Move to the Next Phase

Move to Phase 13 when all 6 baseline algorithms run cleanly through the unified execution interface.

PHASE 13 — Experiments

1. What You Need to Learn

- Matrix experiment automation ($5\ \text{Workloads} \times 6\ \text{Baselines} \times 5\ \text{Seeds} = 150\ \text{Runs}$).
- Automated cluster state resetting between experiment runs.
- Structured result logging (CSV/JSON) with microsecond timestamps.

2. Why You Need It

Automated experiment orchestration ensures rigorous, unbiased, and reproducible empirical data collection.

3. Exact Practical Task

1. Implement `experiments/run_experiment.py` orchestrating workload generation, baseline execution, and CSV output generation.
2. Implement `scripts/reset_cluster.py` to clear pods and metrics between runs.
3. Write master orchestration script `scripts/run_all_experiments.sh`.

4. Expected Output

Automated test harness executing the 150-run matrix and saving raw trace files to `results/raw/`.

5. How to Verify It Works

Execute a mini-matrix test (1 Workload × 2 Baselines × 2 Seeds = 4 runs):

```
python experiments/run_experiment.py --mini-test
```

Verify that 4 non-empty CSV log files appear in `results/raw/`.

6. Common Errors

- **Disk space exhaustion:** Raw logging every second generate large files. Store summary vectors per time-step.
- **Orphaned processes:** Ensure termination signals (`SIGTERM`) clean up background workload generator threads between runs.

7. Relevant GitHub Repositories

- `pytest-dev/pytest` — Automated Python test execution.

8. When to Move to the Next Phase

Move to Phase 14 when the full automated experiment script completes unattended and populates `results/raw/`.

PHASE 14 — Results and Paper

1. What You Need to Learn

- Data processing with Pandas (`read_csv`, `groupby`, `quantile`).
- Data visualization with Matplotlib and Seaborn (Pareto scatter plots, time-series line charts, grouped bar charts).
- Statistical analysis: Percentiles (P95, P99), Cold-Start Rate (CSR), SLA Violation Rate (SVR), non-parametric Wilcoxon signed-rank significance testing (`scipy.stats.wilcoxon`).

2. Why You Need It

Transforms raw execution traces into publication-quality figures, statistical tables, and empirical proof supporting paper research claims.

3. Exact Practical Task

1. Implement `scripts/plot_results.py` reading raw CSV data from `results/raw/`.
2. Compute mean, median, standard deviation, 95% confidence intervals, and p-values.
3. Generate paper figures saved to `results/plots/`:
    - Figure 1: Latency vs. Memory Pareto Frontier.
    - Figure 2: Cold Start Rate across Workloads.
    - Figure 3: Temporal Adaptation Tracking under Bursty Workload (W_3).
    - Figure 4: Ablation Study Cost Attribution (RQ1–RQ3).

4. Expected Output

High-resolution vector plots (`.pdf`/`.png`) and statistical summary tables verifying GranuWarm research claims.

5. How to Verify It Works

Run plot generation script:

```
python scripts/plot_results.py
```

Open generated plots in `results/plots/`. Confirm labels, legend text, and scales are clear and legible. Confirm Wilcoxon test returns $p < 0.05$ for key comparative metrics.

6. Common Errors

- **Calculating mean instead of quantiles for tail latency:** Always use `.quantile(0.95)` and `.quantile(0.99)` for P95/P99 latency calculations.
- **Overlapping plot labels:** Set proper figure dimensions and call `plt.tight_layout()`.

7. Relevant GitHub Repositories

- `pandas-dev/pandas` — Data analysis library.
- `matplotlib/matplotlib` — Visualization library.
- `mwaskom/seaborn` — Statistical data visualization.

8. When to Move to Final Paper Submission

You are done! All software, experimental data, and publication figures are complete and verified.